CSC4093: Neural Networks and Deep Learning

Programming Assignment 01 — Tweet Classification using LSTM Networks

Student Name: Mohammadhu Umair
Student ID: S/20/534

1. Introduction

This notebook implements two LSTM based deep learning models to classify tweets from the PHM dataset as either personal health mentions (label 1) or non-personal health mentions (label 0). Two architectures are compared: a Stacked LSTM and a Bidirectional LSTM. The models are evaluated on accuracy, precision, recall and F1 score.

Tweets are informal and noisy, which makes classification harder than standard text. Careful preprocessing and regularization techniques were applied to handle this.

2. Imports

All required libraries are loaded below. TensorFlow/Keras is used for model building, NLTK for stop word removal, and scikit-learn for evaluation metrics.

3. Data Preparation

The PHM dataset is loaded from CSV files. It contains tweets labelled as personal health mentions (1) or non-personal health mentions (0). An 80/20 split is applied on the training data to create a validation set used during model training to monitor generalisation.

```
Train samples : 9991
Test  samples : 3331

Label distribution (train):
label
0    7091
1    2900
Name: count, dtype: int64
```

4. Preprocessing

Tweets are cleaned through the following steps:

- Lowercasing all text
- Removing URLs, @mentions and hashtag symbols
- Removing non-alphabetic characters
- Removing stop words — with an important exception: negation words like *not*, *no*, and *never* are kept. Dropping them would completely reverse the meaning of many health-related phrases. For example, *not feeling well* would become *feeling well*.
- Removing single-character tokens

```
Train size : 7992 | Val size : 1999
```

5. Tokenization and Sequence Padding

A Keras Tokenizer is fitted on training data only to avoid leaking test vocabulary into the model. Each word is mapped to an integer index by frequency. Sequences are padded to the 90th percentile of training lengths rather than the maximum, which keeps the input size reasonable without truncating most tweets.

```
Max sequence length (90th pct) : 15
Vocabulary size                : 11136
```

6. Hyperparameters

These settings were chosen after observing overfitting in early runs where training accuracy climbed above 93% while validation accuracy stagnated near 80%. Increasing dropout and reducing LSTM units helped close that gap.

```
Embedding dim      : 32
LSTM units (L1/L2) : 64 / 32
Dropout            : 0.5
Recurrent dropout  : 0.3
Batch size         : 128
Max epochs         : 20
L2 regularization  : 1e-4
```

7. Model Definitions

7.1 Stacked LSTM

Two LSTM layers are stacked in sequence. The first layer returns its full sequence of hidden states so the second layer has a temporal view of the whole tweet. SpatialDropout1D drops entire embedding feature dimensions which is more effective than element-wise dropout for sequence data. L2 regularization is added to both LSTM kernels and the Dense layer.

```
C:\Users\USER\AppData\Roaming\Python\Python313\site-packages\keras\src\layers\core\embedding.
py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

Model: "sequential_11"

Layer (type)	Output Shape	Param #
embedding_11 (Embedding)	?	0 (unbuilt)
spatial_dropout1d_11 (SpatialDropout1D)	?	0
lstm_22 (LSTM)	?	0 (unbuilt)
lstm_23 (LSTM)	?	0 (unbuilt)
dense_22 (Dense)	?	0 (unbuilt)
dropout_11 (Dropout)	?	0
dense_23 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

7.2 Bidirectional LSTM

Each LSTM layer is wrapped in a Bidirectional layer so the sequence is processed both forward and backward. The outputs of both directions are concatenated at each time step, doubling the contextual information available to the model. This is particularly useful for health mention tweets where meaning often depends on words that appear later in the sentence.

Model: "sequential_12"

Layer (type)	Output Shape	Param #
embedding_12 (Embedding)	?	0 (unbuilt)
spatial_dropout1d_12 (SpatialDropout1D)	?	0
bidirectional_10 (Bidirectional)	?	0 (unbuilt)
bidirectional_11 (Bidirectional)	?	0 (unbuilt)
dense_24 (Dense)	?	0 (unbuilt)
dropout_12 (Dropout)	?	0
dense_25 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

8. Callbacks

EarlyStopping stops training when validation loss does not improve for 5 consecutive epochs and restores the weights from the best epoch.

ReduceLROnPlateau halves the learning rate when validation loss stalls for 2 epochs, allowing the optimiser to converge more carefully in later training stages.

9. Training

9.1 Stacked LSTM

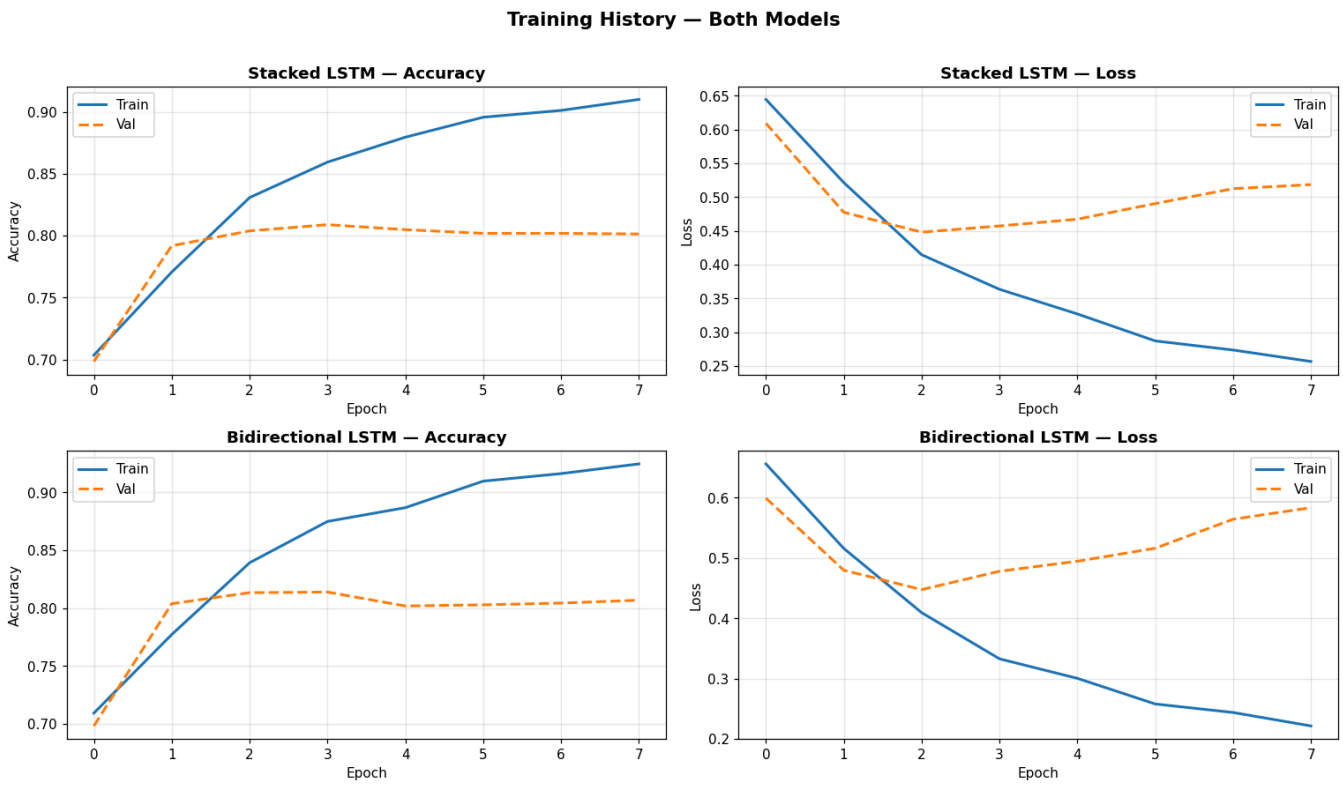
```
Epoch 1/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 8s 50ms/step - accuracy: 0.7035 - loss: 0.6445 - val_accuracy: 0.6
983 - val_loss: 0.6092 - learning_rate: 0.0010
Epoch 2/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 3s 40ms/step - accuracy: 0.7706 - loss: 0.5216 - val_accuracy: 0.7
919 - val_loss: 0.4776 - learning_rate: 0.0010
Epoch 3/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 2s 39ms/step - accuracy: 0.8308 - loss: 0.4148 - val_accuracy: 0.8
039 - val_loss: 0.4482 - learning_rate: 0.0010
Epoch 4/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 2s 38ms/step - accuracy: 0.8595 - loss: 0.3637 - val_accuracy: 0.8
089 - val_loss: 0.4574 - learning_rate: 0.0010
Epoch 5/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.8770 - loss: 0.3333
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
63/63 ━━━━━━━━━━━━━━━━━━━━ 2s 37ms/step - accuracy: 0.8796 - loss: 0.3273 - val_accuracy: 0.8
049 - val_loss: 0.4674 - learning_rate: 0.0010
Epoch 6/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 2s 38ms/step - accuracy: 0.8958 - loss: 0.2874 - val_accuracy: 0.8
019 - val_loss: 0.4905 - learning_rate: 5.0000e-04
Epoch 7/20
62/63 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.8982 - loss: 0.2860
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
63/63 ━━━━━━━━━━━━━━━━━━━━ 3s 39ms/step - accuracy: 0.9013 - loss: 0.2739 - val_accuracy: 0.8
019 - val_loss: 0.5125 - learning_rate: 5.0000e-04
Epoch 8/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 3s 45ms/step - accuracy: 0.9102 - loss: 0.2570 - val_accuracy: 0.8
014 - val_loss: 0.5185 - learning_rate: 2.5000e-04
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 3.
```

9.2 Bidirectional LSTM

```
Epoch 1/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 16s 91ms/step - accuracy: 0.7095 - loss: 0.6561 - val_accuracy: 0.
6983 - val_loss: 0.5993 - learning_rate: 0.0010
Epoch 2/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 5s 80ms/step - accuracy: 0.7773 - loss: 0.5160 - val_accuracy: 0.8
039 - val_loss: 0.4798 - learning_rate: 0.0010
Epoch 3/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 6s 92ms/step - accuracy: 0.8392 - loss: 0.4095 - val_accuracy: 0.8
134 - val_loss: 0.4477 - learning_rate: 0.0010
Epoch 4/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 6s 89ms/step - accuracy: 0.8749 - loss: 0.3330 - val_accuracy: 0.8
139 - val_loss: 0.4781 - learning_rate: 0.0010
Epoch 5/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step - accuracy: 0.8922 - loss: 0.2883
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
63/63 ━━━━━━━━━━━━━━━━━━━━ 6s 88ms/step - accuracy: 0.8868 - loss: 0.3006 - val_accuracy: 0.8
019 - val_loss: 0.4947 - learning_rate: 0.0010
Epoch 6/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 6s 91ms/step - accuracy: 0.9097 - loss: 0.2581 - val_accuracy: 0.8
029 - val_loss: 0.5163 - learning_rate: 5.0000e-04
Epoch 7/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step - accuracy: 0.9180 - loss: 0.2385
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
63/63 ━━━━━━━━━━━━━━━━━━━━ 5s 83ms/step - accuracy: 0.9162 - loss: 0.2440 - val_accuracy: 0.8
044 - val_loss: 0.5644 - learning_rate: 5.0000e-04
Epoch 8/20
63/63 ━━━━━━━━━━━━━━━━━━━━ 5s 86ms/step - accuracy: 0.9245 - loss: 0.2217 - val_accuracy: 0.8
069 - val_loss: 0.5835 - learning_rate: 2.5000e-04
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 3.
```

10. Training History

The plots below show accuracy and loss over epochs for both models side by side. When regularization is working well, training and validation curves should move together rather than diverging.



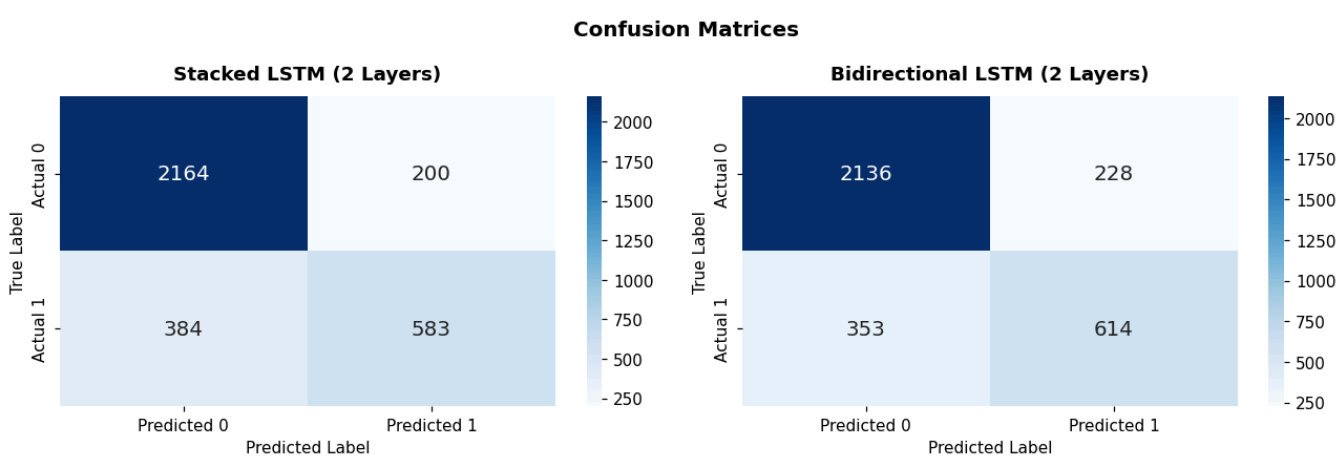
11. Evaluation

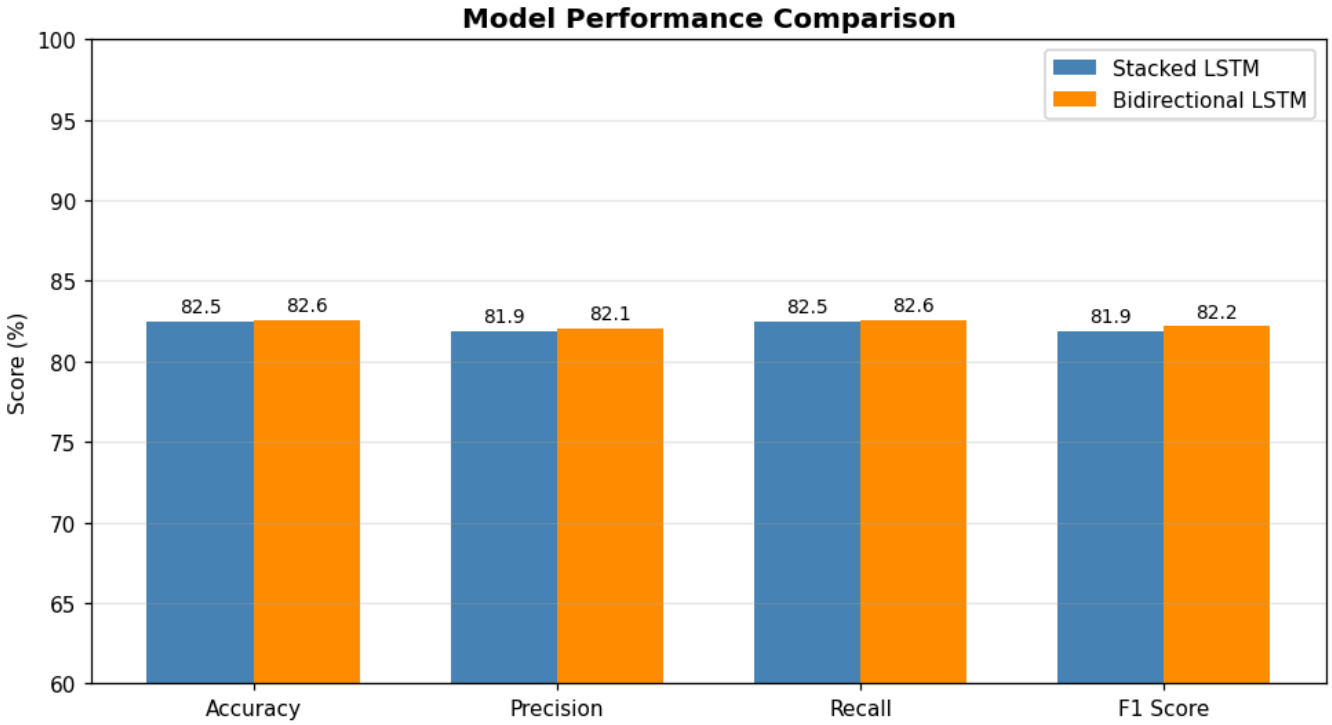
Both models are evaluated on the held-out test set. The confusion matrices are displayed side by side for easy comparison. Precision, recall and F1 score are also printed to give a fuller picture of performance beyond accuracy alone.

```
=== Stacked LSTM (2 Layers) ===
Accuracy  : 82.47%
Precision : 0.8189
Recall    : 0.8247
F1 Score  : 0.8187

              precision    recall  f1-score   support

           0       0.85      0.92      0.88      2364
           1       0.74      0.60      0.67       967

    accuracy                           0.82      3331
   macro avg       0.80      0.76      0.77      3331
weighted avg       0.82      0.82      0.82      3331

--------------------------------------------------
=== Bidirectional LSTM (2 Layers) ===
Accuracy  : 82.56%
Precision : 0.8207
Recall    : 0.8256
F1 Score  : 0.8218

              precision    recall  f1-score   support

           0       0.86      0.90      0.88      2364
           1       0.73      0.63      0.68       967

    accuracy                           0.83      3331
   macro avg       0.79      0.77      0.78      3331
weighted avg       0.82      0.83      0.82      3331

--------------------------------------------------
```

12. Confusion Matrices

The confusion matrices below show the breakdown of correct and incorrect predictions for each model on the test set.



13. Performance Comparison

The bar chart below gives a visual comparison of all four metrics across both models.